

My Redux Toolkit Learning Journey

A beginner's documentation — written while building an Expense Tracker app

Introduction

Before this challenge, I had never touched Redux or Redux Toolkit. I only knew React's `useState` and `useContext`, and honestly, that's all I ever thought I'd need. This documentation is basically my notes-to-future-self on what I learned, how I understood it, and what surprised me while building a small Expense Tracker project (I named it "Trackify") to actually use these concepts instead of just reading about them.

I'm not writing this as an expert. I'm writing this as someone who *just* figured this out, so if future-me (or anyone else reading this) is confused about something, I want this to actually help, not sound like a copy-pasted textbook definition.

Why Redux Toolkit? (a.k.a. Why Not Just `useState`?)

Before this project, I used `useState` everywhere because that's all I knew. While making this expense tracker, I noticed that when I add or delete a transaction, many parts of the app need to update — like the balance cards, the chart, and the transaction list. I could technically do that with `useState` too, passing data down as props or lifting state up, but it started feeling messy really fast. Every component needed to know about the same data, and I'd have had to keep passing it around.

After learning Redux Toolkit, I understood that instead of storing the same data in different places (or passing it through five components just so the sixth one can use it), I can keep it in **one place**, and every component that needs it can just grab it directly. That single idea — "one source of truth" — made a lot more sense to me once I actually ran into the problem myself instead of just reading about it.

In short: `useState` is great for state that only one component cares about. Redux Toolkit is for state that a *lot* of components care about, and that needs to stay in sync everywhere.

What is Redux, Really?

From what I understood: Redux is a way to manage the "state" of your whole app in one central place, instead of scattering it across components. It's not tied to React specifically — it's a general state management pattern, and React-Redux is just the library that connects it to React.

Redux Toolkit (RTK) is basically the "official, modern, less painful" way of writing Redux. From what I read in the docs, plain Redux used to need a lot of boilerplate — action types,

action creators, manually writing immutable updates — and Redux Toolkit was introduced to cut all of that down and make it feel much simpler to write.

Core Concepts (In My Own Words)

Store

The Store is like the single big container that holds all the app's state. In my project, I created it in `store.js` using `configureStore()`. Anything I want available across my app — like my transactions, currency, or username — lives here.

Slice

Honestly, I was confused by the word "slice" at first. 🤔 After actually using it in my project, I think of a slice as **one part of my app's state**, bundled together with the logic that changes it. Since my project is an expense tracker, I made a `transactionsSlice` because all the transaction-related logic (adding, deleting, editing, setting currency, setting username) belongs together in one place.

If I had a different feature — like proper authentication stored in Redux instead of just Context — I could make a separate `authSlice` for that. It just helps keep everything organized instead of dumping all my logic into one giant messy file.

`createSlice()` is the function that lets me define a slice — I give it a name, an initial state, and a bunch of reducer functions, and it automatically generates the action creators for me. That part genuinely saved me a lot of manual work compared to what I saw "old" Redux required.

Reducers

A reducer is just a function that says "given the current state and an action, here's the new state." What surprised me is that with Redux Toolkit, I can write code that *looks* like I'm directly mutating the state (like `state.transactions.push(...)`) even though under the hood it's not actually mutating anything — RTK uses a library called Immer to handle that safely. In plain Redux, I read that you're not supposed to mutate state directly at all, so this was one of those "wait, that's actually allowed here?" moments.

Actions

An action is basically a message that says "hey, something happened, here's what and here's the data." When I dispatch `addTransaction(transactionData)`, that creates an action object behind the scenes with a type and my data as payload, and sends it to the reducer to handle.

useSelector and useDispatch

These two are the bridge between my React components and the Redux store.

- `useSelector` — lets a component **read** data from the store. In my `Dashboard.jsx`, I use it like `useSelector((state) => state.transactions)` to pull out the transactions and currency.
- `useDispatch` — lets a component **send** actions to the store to update it. So when I submit my "Add Transaction" form, I call `dispatch(addTransaction(formData))`.

Basically: `useSelector` = reading, `useDispatch` = writing.

What Happens When I Click "Add Transaction"? (Data Flow)

Earlier I thought clicking the button just adds the transaction to the list. But after building this project, I understood that a few things actually happen in the background:

1. First, the form checks if the entered data is valid (description isn't empty, amount is a real positive number).
2. Once it's valid, it builds a transaction object (with category, icon, color, formatted date, etc.) and sends an **action** to Redux by calling `dispatch(addTransaction(...))`.
3. Redux Toolkit runs my `addTransaction` reducer inside the slice, which updates the state (pushes the new transaction into the array).
4. Redux then notifies every component that's subscribed to that piece of state through `useSelector`.
5. Those components automatically re-render with the latest data — so the balance card, the total income/expense cards, the chart, and the transaction table all update **without me manually telling each one to refresh**.

That last part was honestly the part that surprised me the most. I didn't have to manually update the cards, chart, or table individually — I just updated the *state*, and everything reading from that state updated itself.

Visual (simplified):

```
User submits form
    ↓
Form validates data
    ↓
dispatch(addTransaction(data))
    ↓
Reducer in transactionsSlice.js updates the store
    ↓
Store notifies all subscribed components (via useSelector)
    ↓
Dashboard, Chart, Table all re-render automatically
```

Folder Structure (My Project)

```
src/
├── store/
│   ├── store.js           → creates the Redux store
│   └── transactionsSlice.js → all transaction state + reducers live here
├── components/
│   ├── dashboard/        → components that read/write transaction data
│   ├── layout/           → navbar, sidebar
│   └── settings/         → profile, currency, appearance, danger zone
├── contexts/            → theme, auth, toast (kept separate from Redux
on purpose)
├── pages/               → Dashboard, Settings, Login, Register
└── main.jsx             → wraps the whole app with <Provider store=
{store}>
```

I kept things like theme and auth in React Context instead of Redux, because they felt more like "local app preferences" than shared data that many unrelated components need constantly. This was a judgment call — Redux Toolkit docs don't force you to put *everything* in the store, only what actually needs to be shared.

Important Functions I Used

Function	What it does (in my words)
<code>configureStore()</code>	Creates the Redux store, wires up my reducers, sets up good defaults (like DevTools) automatically
<code>createSlice()</code>	Defines a slice — state + reducers — and auto-generates action creators
<code>useSelector()</code>	Reads a piece of state from the store inside a component
<code>useDispatch()</code>	Gives me a function to send (dispatch) actions to the store
<code>nanoid()</code>	A small utility from RTK to generate unique IDs — I used this for each transaction's id

Real-World Use Cases

From what I understood, Redux Toolkit makes the most sense in apps where:

- The same data is needed in many unrelated components (like my balance showing up in cards, chart, and table)
- State needs to persist and stay consistent across page navigation

- There's a lot of shared logic for updating that data (like add/edit/delete transactions)

Examples I thought of: shopping cart apps, dashboards, social media feeds (likes/comments state), or any app with a logged-in user's data used across many screens.

Challenges I Faced

(Fill this in with what actually confused or slowed you down — e.g. understanding immutability, connecting the Provider, debugging a reducer not updating, etc. This is the part that'll make your video and LinkedIn post feel real, so it's worth a genuine 2-3 minutes of thinking back on your own process.)

Additional Things I Explored

(Anything extra you looked into beyond the core requirement — for example, you also built localStorage syncing, theming, filtering/search, and a chart with Chart.js. Write a couple lines on why you added these and what you learned setting them up.)

Final Thoughts

Going into this, "Redux" sounded intimidating — like something only senior developers used. But building an actual project with it made the core idea click fast: keep shared data in one place, read it with `useSelector`, update it with `useDispatch`, and let React handle the re-rendering. The boilerplate I was scared of turned out to be mostly solved by Redux Toolkit itself.

I still have more to learn — this project didn't touch async actions (`createAsyncThunk`) or RTK Query, so that's probably my next stop.